

Programowanie obiektowe

Wykład 2

Definiowanie klas

Obiekt i klasa

- Obiekt to **samodzielna jednostka** w programie, która reprezentuje **rzecz, pojęcie lub element systemu**. Może posiadać **własny stan** (przechowywany w atrybutach) oraz **zachowanie** (określone przez metody). Obiekty mogą **współpracować** ze sobą, przekazywać dane i reagować na zmiany w programie.
 - Przechowuje dane i wykonuje operacje.
 - Może oddziaływać na inne obiekty.
 - Może być częścią złożonych struktur.
- Klasa to **definicja, wzór lub szablon**, według którego tworzone są obiekty. Określa, **jakie właściwości i zachowania** będą mieć obiekty danej klasy. W programie obiektowym klasy pomagają w organizacji kodu, ponieważ grupują logicznie powiązane dane i operacje w jednym miejscu.
 - Umożliwia tworzenie wielu obiektów o tych samych właściwościach, ale różnych wartościach.
 - Raz zdefiniowana klasa może być używana w różnych częściach programu.

Obiekt i klasa – przykład

- **Klasa:** *Punkt* – definiuje, co charakteryzuje każdy punkt. Opisuje jego cechy i zachowania.
 - Atrybuty, czyli dane przechowywane w obiekcie, określające jego aktualny stan (np. jego współrzędne `x` i `y`).
 - Operacje, które można na nim wykonać (np. przesunięcie).
 - Nie jest jeszcze konkretnym punktem, ale jego definicją.
- **Obiekt:** Konkretna instancja klasy – *rzeczywisty punkt*, który istnieje w programie. Możemy mieć wiele obiektów klasy *Punkt*, a każdy z nich może mieć inne wartości współrzędnych.

 Point
□ <code>x: float</code> □ <code>y: float</code>
● <code>move(dx: float, dy: float)</code> ● <code>display()</code>

p1
<code>x = 2.5</code> <code>y = 3.0</code>

p2
<code>x = -1.0</code> <code>y = 4.5</code>

p3
<code>x = 0.0</code> <code>y = 0.0</code>

Punkt w programie strukturalnym

W programowaniu strukturalnym, nie mając klas i obiektów, programista sam musi zarządzać danymi i operacjami na nich.

Jak powinniśmy reprezentować punkt?

Jako dwie zmienne?

```
float x = 3.5;  
float y = 2.0;
```

Jako tablicę?

```
float punkt[2] = {3.5, 2.0};
```

Jako strukturę?

```
struct Punkt {  
    float x;  
    float y;  
};  
struct Punkt p = {3.5, 2.0};
```

Punkt w programie strukturalnym

Jak wykonywać operacje na punkcie?

Bezpośrednio manipulować zmiennymi?

```
x += 1.5;  
y -= 0.5;
```

Napisać funkcje do operacji na danych?

```
void przesunPunkt(struct Punkt *p, float dx, float dy) {  
    p->x += dx;  
    p->y += dy;  
}  
  
// W kodzie main():  
struct Punkt p = {3.5, 2.0};  
przesunPunkt(&p, 1.5, -0.5);
```

Problem: **reprezentacja** – w podejściu strukturalnym musimy pamiętać o reprezentacji, w podejściu obiektowym *punkt* jest reprezentowany przez obiekt klasy *Punkt*.

Abstrakcja

Jeden z czterech filarów programowania obiektowego (obok hermetyzacji, dziedziczenia i polimorfizmu).

Abstrakcja: wyodrębnianie istotnych cech i zachowań obiektu, pomijanie detali zbędnych w danym kontekście.

Punkt w programie jest reprezentowany przez **obiekt**.

- Obiekt zawiera pewne **dane** (określające jego stan).
- Jeszcze ważniejsze jest, że obiekt oferuje pewne **operacje**.

Obiekt jako serwer usług.

- Udostępnia swoje metody jako usługi dla innych obiektów.
- Zachowuje kontrolę nad swoimi danymi.

 Point
□ x: float □ y: float
● move(dx: float, dy: float) ● display()

Wykorzystanie obiektu w programie

Dobłą praktyką jest rozpoczęcie pisania programu od kodu klient lub testów.

- Definiujemy **interfejs** klasy zanim napiszemy jej **implementację**, co pomaga w jej lepszym zaprojektowaniu.
- Skupiamy się na tym, **jak klasa ma być używana**, zamiast od razu wchodzić w szczegóły techniczne.
- Unikamy zbędnej komplikacji, bo piszemy tylko to, co jest potrzebne **klientowi/testowi**.

Czym jest **klient**?

- To kod lub obiekt, który używa innej klasy, wywołując jej metody. Może to być np. inna klasa, moduł lub część aplikacji.

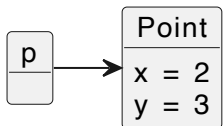
```
// Main.java
public class Main {
    public static void main(String[] args) {
        Point p = new Point(2, 3); // Tworzymy obiekt (choć jeszcze nie mamy klasy!)
        p.move(1, -1);             // Oczekujemy, że ta metoda przesunie punkt
        p.display();               // Oczekiwane: Punkt (3.0, 2.0)
    }
}
```

Obiekt a referencja

```
Point p = new Point(2, 3);
```

Ta linijka kodu tworzy nowy **obiekt** klasy `Point` i przypisuje go do **referencji** `p`.

- `Point p` - deklaruje **referencję** `p` typu `Point`, ale jeszcze nie przypisuje do niej żadnego **obiektu**.
- `new Point(2, 3)` - tworzy nowy **obiekt** `Point` w pamięci dynamicznej (na stercie).
- `=` - Przypisuje adres nowo utworzonego **obiektu** do **referencji** `p`.



Tak samo działają obiekty dynamiczne (tworzone na stercie) w **C++**:

```
Point* p = new Point(2, 3);  
p->move(1, -1);  
// ...  
delete p;
```


Definiowanie klasy

Kod klienta zakłada istnienie klasy i pozwala nam określić oczekiwania wobec niej.

```
Point p = new Point(2, 3);  
p.move(1, -1);  
p.display();
```

Ale aby móc stworzyć **obiekt**, należy zdefiniować **klasę**:

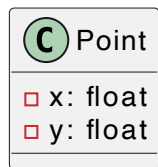
```
// Definicja klasy Point  
class Point {  
    // miejsce na składowe klasy (pola, metody, etc.)  
}
```

Konwencje nazewnnicze:

- **PascalCase** dla nazw klas, czyli każda pierwsza litera wyrazu jest wielka (np. `BankAccount`).
- Rzeczowniki lub frazy rzeczownikowe (np. `User` , `InvoiceManager`).
- Bez skrótów (`DatabaseConnection` zamiast `DBConn`).

Pola klasy

- **Pola klasy** to zmienne należące do klasy, które przechowują dane każdego obiektu. To one określają, jakie właściwości ma obiekt.
- **Stan obiektu** to zbiór wartości wszystkich jego pól w danym momencie działania programu.
- Stan może się zmieniać (np. `pałiwo` w klasie `Samochód`) lub też nie (np. `marka` w klasie `Samochód`), ale zawsze wynika z wartości pól.



p1
x = 2.5
y = 3.0

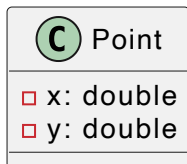
p2
x = -1.0
y = 4.5

p3
x = 0.0
y = 0.0

Pola (wraz z metodami) są **składowymi klasy**.

Definiowanie pól klasy

- **Pola klasy** są zmiennymi zadeklarowanymi wewnątrz klasy (*ale poza metodami, konstruktorami czy blokami inicjalizacyjnymi*).



W Javie preferowane jest użycie typu `double`.

```
class Point {  
    // definicje pól klasy  
    double x;  
    double y;  
}
```

Konwencje nazewnnicze:

- **camelCase** – zaczynają się małą literą, a pierwsza litera każdego kolejnego wyrazu jest wielka (np. `balance`, `maxSpeed`, `isEngineRunning`, `totalPrice`).

Metody

Dysponując kodem klienta, łatwo określić wymagane metody (w tym ich nagłówki). **Z punktu widzenia klienta, metody są najważniejsze.**

```
p.move(1, -1);  
p.display();
```

Metody są funkcjami zdefiniowanymi wewnątrz klasy.

```
class Point {  
    ...  
    void move(double dx, double dy) {  
        ...  
    }  
    void display() {  
        ...  
    }  
}
```

Konwencje nazewnnicze

- Java i C++ używają **camelCase** (`calculateSalary()`), natomiast C# **PascalCase** (`CalculateSalary()`).
- Nazwy czasownikowe, określające działanie (`saveFile()` , `GetUserData()`).

Definiowanie metod

Metoda operuje na stanie obiektu.

```
class Point {  
    double x;  
    double y;  
  
    void move(double dx, double dy) {  
        x += dx;  
        y += dy;  
    }  
    void display() {  
        System.out.println("Punkt (" + x + ", " + y + ")");  
    }  
}
```

W ciele metody możemy sięgać do stanu obiektu (wartości pól), aby go odczytywać lub modyfikować.

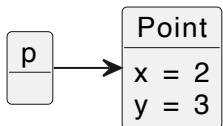
Metody (wraz z polami) są **składowymi klasy**.

Wywoływanie metod

Metoda operuje na stanie obiektu.

Metodę wywołujemy z konkretnej instancji klasy.

```
// kod klienta  
Point p = new Point(2, 3)
```



```
class Point {  
    double x;  
    double y;  
  
    void move(double dx, double dy) {  
        x += dx;  
        y += dy;  
    }  
}
```

I operuje ona na stanie tej konkretnej instancji.

```
p.move(1, -1);
```



Kontekst metody

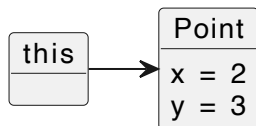
Metoda działa w kontekście instancji z której została wywołana.

```
p.move(1, -1);
```



W metodach instancyjnych **do bieżącego kontekstu możemy sięgnąć przy pomocy** `this`.

```
class Point {  
    double x;  
    double y;  
  
    void move(double dx, double dy) {  
        this.x += dx;  
        this.y += dy;  
    }  
}
```



Czym jest `this`?

```
class Point {  
    ...  
    void move(double dx, double dy) {  
        this.x += dx;  
        this.y += dy;  
    }  
}
```

`this` jest **jawną referencją do bieżącego obiektu**, czyli obiektu, w kontekście którego działa metoda instancyjna. Pozwala nam m.in. odwoływać się do pól instancji.

Niekiedy jest wymagane, aby jednoznacznie wskazać, czy odnosimy się do pola klasy, czy np. parametru lub zmiennej lokalnej.

```
class Point {  
    ...  
    void setX(double x) {  
        this.x = x;  
    }  
}
```


Tworzenie obiektów

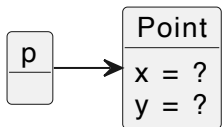
```
Point p = new Point(2, 3);  
p.move(1, -1);  
p.display();
```

Aby powyższy kod zadziałał, brakuje nam jeszcze jednego: `new Point(2, 3)`.

Na razie możemy utworzyć *punkt* jedynie w ten sposób:

```
Point p = new Point();
```

Gdy tworzymy obiekt, rezerwowana jest na niego pamięć. W tym momencie powstają wszystkie **pola** instancji.



Ale co z ich **inicjalizacją**?

Konstruktor

Konstruktor służy do inicjalizacji obiektu.

Konstruktor to specjalny blok kodu, który jest automatycznie wywoływany w momencie tworzenia obiektu.

Jego rolą jest **inicjalizacja** obiektu (ale NIE jego **stworzenie** – gdy wykonywany jest konstruktor, obiekt już istnieje).

```
class Point {  
    Point() {  
        x = 2;  
        y = 3;  
    }  
}
```

Konstruktor **nie jest metodą**, choć ma wiele cech wspólnych z metodami.

- Oba mają ciało `{ }` z instrukcjami.
- Oba mogą mieć parametry.
- Konstruktor **nie ma typu zwracanego**, metoda tak.
- Konstruktor **nazywa się jak klasa**, metody mogą mieć dowolną nazwę.
- Konstruktor **jest wywoływany automatycznie** (gdy tworzymy obiekt), metoda jawnie.

Parametry konstruktora

```
Point p = new Point(2, 3);
```

Aby móc określać początkowy stan obiektu w momencie (i w miejscu) jego utworzenia, powinniśmy zdefiniować konstruktor z parametrami.

```
class Point {  
    double x;  
    double y;  
  
    Point(double x, double y) {  
        this.x = x;  
        this.y = y;  
    }  
}
```

W tym wypadku użycie `this` jest niezbędne, aby było jasne czy sięgamy do pola klasy (`this.x`), czy do parametru (`x`).

Konstruktor domyślny

Szczególny przypadek – konstruktor bezparametrowy.

Jeśli w klasie **nie zdefiniujemy żadnego konstruktora**, kompilator automatycznie **tworzy konstruktor domyślny**.

Konstruktor domyślny to **konstruktor bez parametrów**, który inicjalizuje obiekt **wartościami domyślnymi**.

```
class Point {  
    double x;  
    double y;  
  
    Point() {  
        x = 0.0;  
        y = 0.0;  
    }  
}
```

Wartość domyślna zależy od typu danych:

- Dla typów prostych: `0` lub `0.0` dla typów liczbowych, pusty znak `'\u0000'` dla `char`, `false` dla `boolean`.
- Dla typów referencyjnych (tablice, obiekty, w tym `String`) – wartość `null`.

Brak konstruktora domyślnego

```
class Point {  
    double x;  
    double y;  
  
    Point(double x, double y) {  
        this.x = x;  
        this.y = y;  
    }  
}
```

Jeśli w klasie istnieje choć jeden **konstruktor z parametrami**, kompilator **NIE tworzy** domyślnego konstruktora.

```
Point p = new Point(); // ❌ Błąd kompilacji - brak konstruktora domyślnego!
```

Aby uniknąć tego błędu, możemy jawnie dodać konstruktor bezparametrowy.

Inne sposoby inicjalizacji pól

- Inicjalizacja **w miejscu deklaracji**.

```
class Point {  
    String color = "black";  
}
```

- Tylko w **Javie** – **blok inicjalizacyjny** (wywołuje się przed konstruktorem).

```
import java.util.Random;  
class Point {  
    String color;  
    // Blok inicjalizacyjny - losowanie koloru czarny/biały  
    {  
        Random rand = new Random();  
        if (rand.nextBoolean()) { // `nextBoolean()` zwraca `true` lub `false`  
            color = "black";  
        } else {  
            color = "white";  
        }  
    }  
}
```

Java

- W **Javie** klasę `Point` możemy zdefiniować jako publiczną – w pliku o nazwie takiej jak nazwa klasy.

```
// Point.java
public class Point {

}
```

- Lub w tym samym pliku, co główna klasa programu (zawierająca `main()`) - wówczas klasa `Point` nie może być publiczna.

```
// Program.java
class Point {

}
public class Program {
    public static void main(String[] args) {

    }
}
```

- *Uważajmy, aby nie zdefiniować jednej klasy wewnątrz drugiej – jest to możliwe, ale niesie konsekwencje o których pomówimy na kolejnych wykładach.*

C#

- W **C#** składowe, z których korzysta klient, powinny być **publiczne** (poprzedzone słowem kluczowym `public`).
- Więcej na ten temat – na kolejnym wykładzie.

```
class Point {  
    int x;  
    int y;  
  
    public Point(int x, int y) { ... }  
    public void Move(int dx, int dy) { ... }  
    public void Display() { ... }  
}
```

- `Move()` zamiast `move()` czy `Display()` zamiast `display()` to tylko kwestia **konwencji nazewnictwa**.

C++

- W **C++** również wymagane jest oznaczenie **publicznych** składowych, ale składnia jest inna – **public:** rozpoczyna blok publiczny.

```
class Point {  
    int x, y;  
  
public:  
    Point(int x, int y) { ... }  
    void move(int dx, int dy) { ... }  
    void display() { ... }  
};
```

- Uwaga na **wymagany** średnik (;) na końcu definicji klasy.

C++

- W C++ `this` jest **wskaźnikiem**, a nie referencją (jest typu `Point*`).

```
class Point {  
    ...  
public:  
    Point(int x, int y) {  
        this->x = x;  
        this->y = y;  
    }  
};
```

- Zamiast inicjalizowania pól w konstruktorze, częściej stosuje się **listy inicjalizacyjne**.

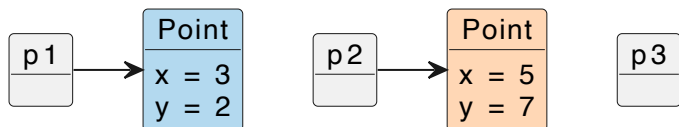
```
class Point {  
    ...  
public:  
    Point(int x, int y) : x(x), y(y) {}  
};
```

- Jest to preferowane, przeważnie wydajniejsze, a w niektórych przypadkach jedyne możliwe rozwiązanie.

Obiekty dynamiczne w Javie i C#

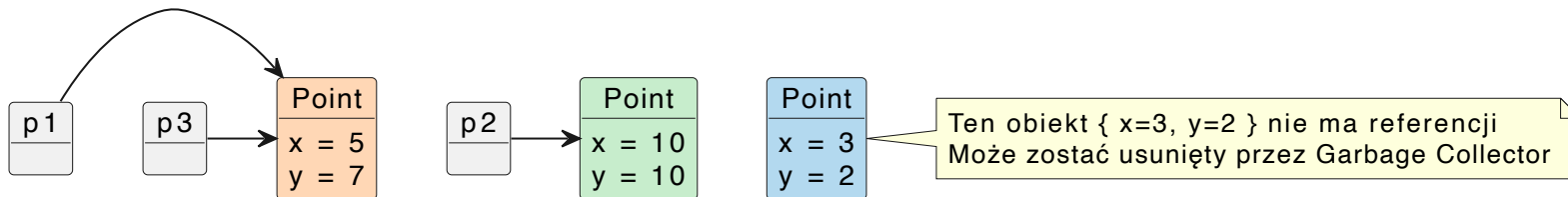
W **Java** i **C#** obiekty są **dynamiczne** (tworzone na **stercie**). Wskazują na nie **referencje**.

```
Point p1 = new Point(3, 2);  
Point p2 = new Point(5, 7);  
Point p3;
```



- Referencja jest zmienną **automatyczną**. Może wskazywać na obiekt, ale nie musi.
- Kilka referencji może wskazywać na ten sam obiekt.
- Jeśli obiekt nie ma referencji, jest usuwany przez **Garbage Collector**.

```
p1 = p2;  
p2 = new Point(10, 10);  
p3 = p1;
```



Interakcja pomiędzy obiektami

Świadomość odrębności poszczególnych instancji jest niezbędna w sytuacjach, gdy w interakcję wchodzi wiele obiektów, np.:

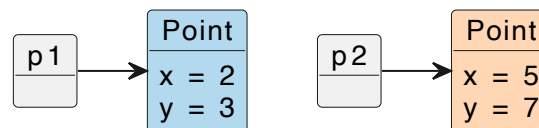
```
Point p1 = new Point(2, 3);
Point p2 = new Point(5, 7);

double distance = p1.distanceTo(p2);
```

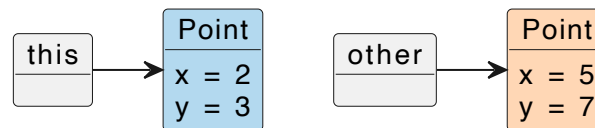
Metoda `distance()` ma obliczyć odległość między dwoma punktami.

```
public class Point {
    ...
    public double distanceTo(Point other) {
        int dx = other.x - this.x;
        int dy = other.y - this.y;
        return Math.sqrt(dx * dx + dy * dy);
    }
}
```

Tak wyglądają te obiekty z punktu widzenia kodu klienta (metoda `main()`):



A tak z punktu widzenia metody `distance()`:



Oczywiście mamy też zmienne lokalne:



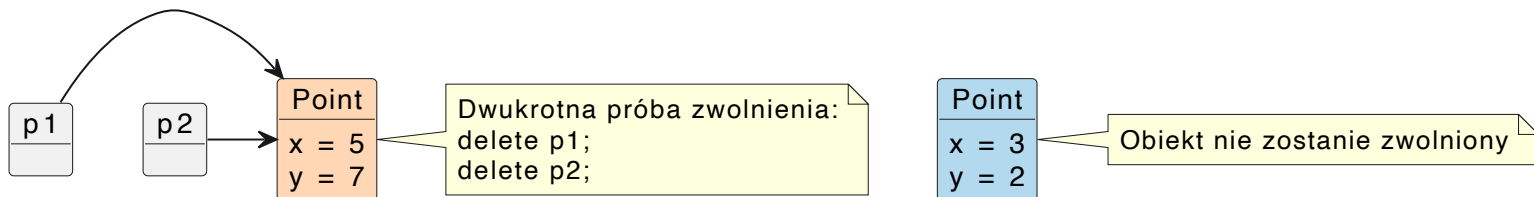
Obiekty dynamiczne w C++

W przeciwieństwie do Javy i C#, w **C++** programista musi zadbać o **zwolnienie** dynamicznych obiektów.

```
Point* p1 = new Point(3, 2);  
p1->display();  
delete p1;
```

Nieostrożne posługiwanie się wskaźnikami, może prowadzić do sytuacji, gdy pewne obiekty nie zostaną zwolnione lub dojdzie do próby ponownego ich zwolnienia.

```
Point* p1 = new Point(3, 2);  
Point* p2 = new Point(5, 7);  
p1 = p2;  
  
delete p1;  
delete p2;
```

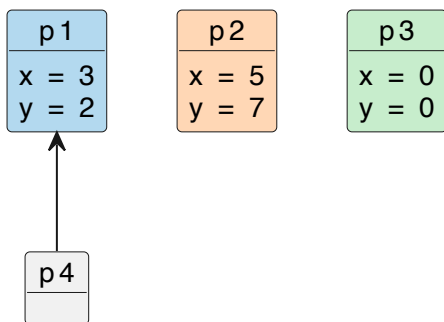


Obiekty automatyczne w C++

Obiektami automatycznymi posługujemy się jak zmiennymi typów podstawowych.

Pamięć na nie jest przydzielana i zwalniana **automatycznie** (na **stosie**).

```
Point p1(3, 2);  
Point p2(5, 7);  
Point p3; // to też jest obiekt - użyliśmy konstruktora bezparametrowego  
  
p1.display();  
p2.display();  
p3.display();  
  
Point &p4 = p1; // referencja na inny obiekt  
p4.display();
```



Destruktory w C++

W C++ szczególną rolę pełnią **destrukторы**.

- Destruktor jest **wywoływany automatycznie** w momencie **usuwania obiektu z pamięci**.
- Działa zarówno dla obiektów automatycznych (gdy wychodzą poza zakres) jak i dynamicznych (gdy użyjemy `delete`).
- Jego główną rolą jest **zwalnianie zasobów** zarezerwowanych przez obiekt (np. pamięci dynamicznej).
- Nazwa destruktora to nazwa klasy poprzedzona tyldą (`~`).
- Nie przyjmuje argumentów i nie zwraca wartości.

```
class Point {  
    ...  
public:  
    Point(double x, double y): x(x), y(y) {}  
    ~Point() {  
        cout << "Destruktor" << endl;  
    }  
};
```

- W tak prostej klasie jak `Point` nie ma zbyt wiele do roboty.

Wprowadzenie do UML

Unified Modeling Language (UML) to standardowy język graficzny używany do wizualizacji, specyfikowania, konstruowania i dokumentowania systemów programistycznych.

UML to język modelowania, który:

- Służy do graficznego przedstawienia struktur i procesów w systemach informatycznych.
- Pomaga w analizie i projektowaniu aplikacji obiektowych.
- Jest niezależny od konkretnego języka programowania (może być stosowany w Pythonie, Javie, C++, itp.).
- Ułatwia komunikację między programistami, analitykami i projektantami systemów.
- Umożliwia dokumentowanie istniejących systemów oraz planowanie nowych rozwiązań.

Podstawowe rodzaje diagramów UML

UML oferuje wiele typów diagramów, m.in.:

- Diagram klas – najważniejszy dla programowania obiektowego; przedstawia klasy, ich atrybuty, metody i relacje między nimi.
- Diagram obiektów – prezentuje konkretną instancję (obiekt) klasy i jej aktualny stan.
- Diagram sekwencji – ilustruje interakcję między obiektami w czasie wykonania programu.
- Diagram przypadków użycia – używany do przedstawienia funkcjonalności systemu z perspektywy użytkownika.

W ramach wykładu kluczowe znaczenie będzie miał diagram klas, ponieważ pozwala na wizualne przedstawienie struktury programu opartego na klasach i obiektach.

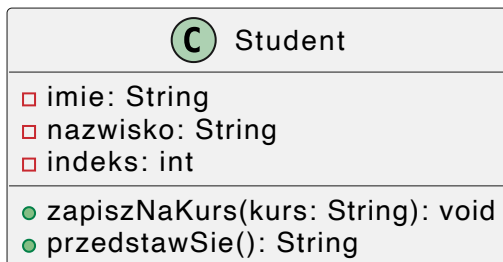
Diagram klas UML – podstawowe elementy

Diagram klas przedstawia klasy, ich atrybuty, metody oraz relacje między nimi.

Każda klasa w UML jest przedstawiona jako prostokąt podzielony na trzy sekcje:

1. **Nazwa klasy** – znajduje się na górze, powinna być zapisana w liczbie pojedynczej i dużą literą.
2. **Atrybuty (pola)** – znajdują się w środkowej sekcji i określają właściwości klasy. Składają się z nazwy i typu danych.
3. **Metody (operacje)** – znajdują się w dolnej sekcji i opisują działania, które klasa może wykonywać. Ich zapis zawiera nazwę oraz opcjonalne parametry wejściowe i typ zwracany.

Przykład dla klasy `Student` :



Relacje między klasami

W programowaniu obiektowym klasy często współpracują ze sobą. UML pozwala modelować różne relacje między klasami, co pomaga lepiej zrozumieć strukturę systemu.

Asocjacja reprezentuje powiązanie między dwiema klasami, wskazujące, że obiekty jednej klasy mogą znać i wykorzystywać obiekty drugiej klasy, np. `Student` i `Uczelnia`.

Może mieć krotność (np. `1`, `0..1`, `*`, `1..*`), określającą liczbę instancji w relacji.

Zależność jest słabszym rodzajem powiązania. Oznacza że jedna klasa korzysta z drugiej w sposób tymczasowy, np. `Student` i `Biblioteka`.

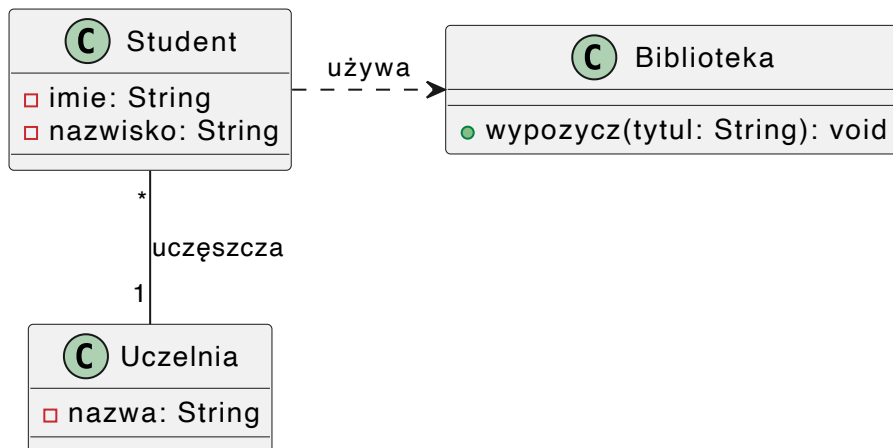
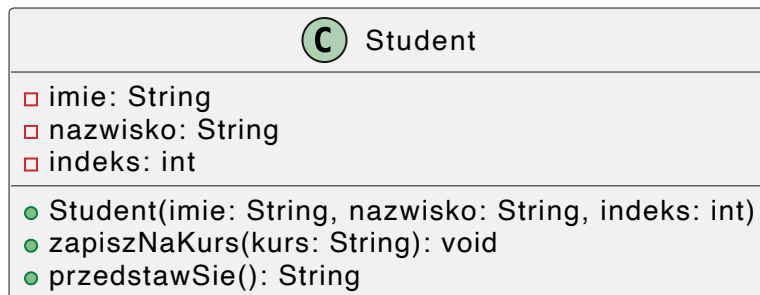


Diagram UML a kod

■ Diagram UML klasy Student



■ Kod w Javie odpowiadający diagramowi UML

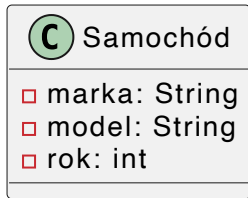
```
public class Student {
    String imie;
    String nazwisko;
    int indeks;

    public Student(String imie, String nazwisko, int indeks) { ... }
    public void zapiszNaKurs(String kurs) { ... }
    public String przedstawSie() { ... }
}
```

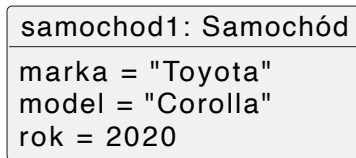
Diagram obiektów UML

Służy do przedstawienia konkretnych instancji klas wraz z ich konkretnymi wartościami atrybutów w określonym momencie działania programu. Jest szczególnie przydatny do analizy struktury danych i stanu obiektów w czasie rzeczywistym.

Przykład – jeśli mamy klasę `Samochód` :



To jej instancję można przedstawić jako obiekt w UML:



Tutaj `samochod1` to konkretna instancja klasy `Samochód` , z przypisanymi wartościami dla atrybutów.

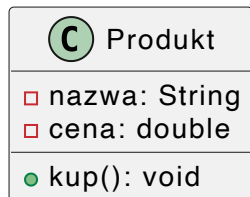
Narzędzia do tworzenia diagramów UML

Istnieje mnóstwo narzędzi do tworzenia diagramów UML, m.in.:

- draw.io – graficzny edytor online.
- PlantUML – prosty język do generowania diagramów tekstowych.

Przykład generowania diagramów w PlantUML:

```
@startuml
class Produkt {
  - nazwa: String
  - cena: double
  + kup(): void
}
@enduml
```



Stos – przykład krok po kroku

Stos to liniowa struktura danych działająca na zasadzie LIFO (Last In, First Out), w której elementy są dodawane na szczyt stosu i z niego zdejmowane.

Zaczynamy od kodu klienta.

```
Stack stack = new Stack(5); // Tworzymy stos o pojemności 5

stack.Push(10);
stack.Push(20);
stack.Push(30);
// Teraz stos zawiera [ 10, 20, 30 ]

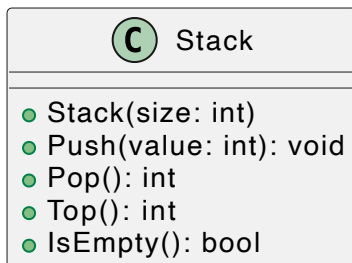
Console.WriteLine("Top: " + stack.Top()); // Powinno pokazać 30
Console.WriteLine("Pop: " + stack.Pop()); // Usuwamy 30
// Stos zawiera [ 10, 20 ]

// Próba usunięcia wszystkich elementów
while (!stack.IsEmpty()) {
    Console.WriteLine("Pop: " + stack.Pop());
}
// Wypisuje 20, a następnie 10
```

Klient posługuje się **abstrakcją** stosu: ważne jest **Co** a nie **Jak**.

Definiujemy klasę

Kod klienta podpowiada nam interfejs klasy.



Na tej podstawie tworzymy zaczątek kodu:

```
class Stack {
    public Stack(int size) { }
    public void Push(int value) { }
    public int Pop() { }
    public int Top() { }
    public bool IsEmpty() { }
}
```

Stos to nie tylko operacje – to klasa:

- łączy dane i operacje,
- ukrywa reprezentację,
- pozwala stworzyć wiele niezależnych instancji
- i najważniejsze: modeluje pojęcie z rzeczywistości.

To moment, gdy powinniśmy zastanowić się nad odpowiedzialnością klasy.

- **Stos** przechowuje liczby; pamięta, która jest na szczycie; dba o utrzymanie spójnego stanu
- **nie** wczytuje ani **nie** wypisuje danych
- **nie** odpowiada za to, co klient z nimi robi

Definiujemy pola

- Wartości odłożone na stosie będziemy przechowywać w standardowej tablicy (`int[]`).
- Poza tablicą na elementy stosu (`items`), potrzebujemy informacji o liczbie odłożonych elementów.
- Tę rolę będzie pełnić pole `top` – indeks wierzchołka stosu.

C Stack
□ <code>items: int[]</code> □ <code>top: int</code>
● <code>Stack(size: int)</code> ● <code>Push(value: int): void</code> ● <code>Pop(): int</code> ● <code>Top(): int</code> ● <code>IsEmpty(): bool</code>

Uwaga: Stos to nie tablica `items` .

Z punktu widzenia klienta, Stos to operacje Push, Pop, Top.

```
class Stack {  
    int[] items;  
    int top;  
  
    public Stack(int size) { }  
  
    public void Push(int value) { }  
    public int Pop() { }  
    public int Top() { }  
    public bool IsEmpty() { }  
}
```

Definiujemy konstruktor

- Konstruktor inicjalizuje stos o określonym rozmiarze (`size`).
 - Tworzymy tablicę o podanej wielkości.
 - Początkowo stos jest pusty, zatem jego wierzchołek inicjujemy na `-1` .

```
top == -1      // stos pusty
0 <= top < size // stos zawiera elementy
```

```
class Stack {
    int[] items;
    int top;

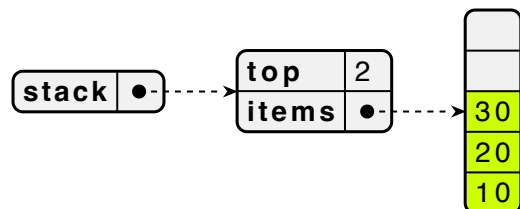
    public Stack(int size) {
        items = new int[size];
        top = -1;
    }

    public void Push(int value) { }
    public int Pop() { }
    public int Top() { }
    public bool IsEmpty() { }
}
```

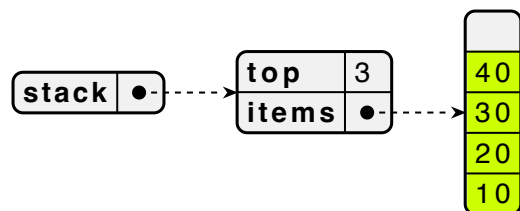
Rolą metod będzie nie tylko wykonywanie manipulacji danymi, ale również utrzymywanie poprawności obiektu (pilnowanie aby nie znalazł się w niepoprawnym stanie).

Definiujemy Push()

- Metoda `Push()` dodaje element na szczyt stosu.
- Zatem jeśli stan stosu był taki:



- Po wykonaniu `stack.Push(40)` będzie taki:

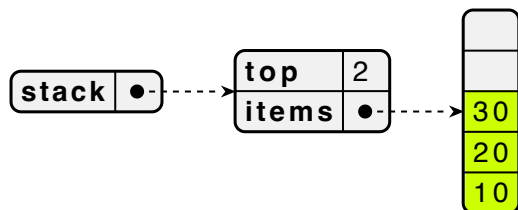


Poza zwiększeniem indeksu i dodaniem elementu, warto sprawdzać czy stos nie jest przepelniony (`top == items.Length - 1`).

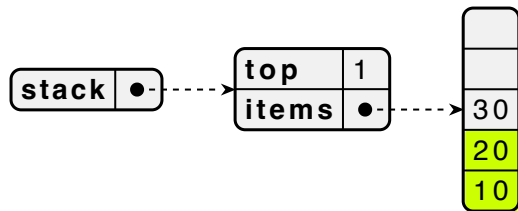
```
class Stack {  
    int[] items;  
    int top;  
  
    public Stack(int size) {  
        items = new int[size];  
        top = -1;  
    }  
  
    public void Push(int value) {  
        items[++top] = value;  
    }  
    public int Pop() { }  
    public int Top() { }  
    public bool IsEmpty() { }  
}
```

Definiujemy Pop()

- Metoda `Pop()` usuwa i zwraca element ze szczytu stosu.
- Jeśli stan stosu był taki:



- Powinniśmy zwrócić wartość spod indeksu 2.
- A następnie zmniejszyć indeks (`top--`).



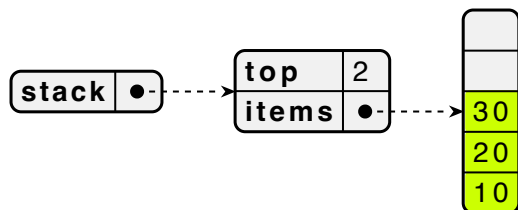
- To nieistotne, że w tablicy pozostała wartość 30.
- O tym gdzie znajduje się szczyt stosu mówi nam pole `top`.

```
class Stack {  
    int[] items;  
    int top;  
  
    public Stack(int size) {  
        items = new int[size];  
        top = -1;  
    }  
  
    public void Push(int value) {  
        items[++top] = value;  
    }  
  
    public int Pop() {  
        return items[top--];  
    }  
  
    public int Top() { }  
    public bool IsEmpty() { }  
}
```

Warto sprawdzać czy stos nie jest pusty (np. przy pomocy `IsEmpty()`).

Definiujemy Top()

- Metoda `Top()` zwraca element ze szczytu stosu, ale go nie usuwa.
- Dla stosu:



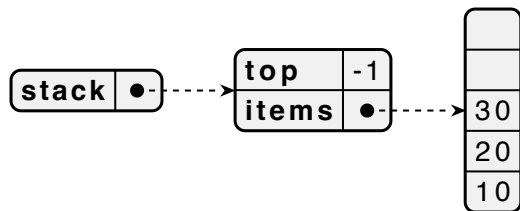
- Zwracamy wartość `30`.
- I nie modyfikujemy stosu.

```
class Stack {  
    int[] items;  
    int top;  
  
    public Stack(int size) {  
        items = new int[size];  
        top = -1;  
    }  
  
    public void Push(int value) {  
        items[++top] = value;  
    }  
    public int Pop() {  
        return items[top--];  
    }  
    public int Top() {  
        return items[top];  
    }  
    public bool IsEmpty() { }  
}
```

Warto sprawdzać czy stos nie jest pusty.

Definiujemy metodę `IsEmpty()`

- Metoda `IsEmpty()` sprawdza, czy stos jest pusty
- TO, czy stos jest pusty, zależy jedynie od wartości `top`, a nie faktycznej zawartości tablicy.



```
class Stack {  
    int[] items;  
    int top;  
  
    public Stack(int size) {  
        items = new int[size];  
        top = -1;  
    }  
  
    public void Push(int value) {  
        items[++top] = value;  
    }  
  
    public int Pop() {  
        return items[top--];  
    }  
  
    public int Top() {  
        return items[top];  
    }  
  
    public bool IsEmpty() {  
        return top == -1;  
    }  
}
```

Pełny kod klasy

- Warto ją uzupełnić o sprawdzanie warunków mogących spowodować nieprawidłowe działanie (np. próba zdjęcia wartości z pustego stosu).
- Ponieważ rozmiar stosu jest ograniczony warto dodać metodę sprawdzającą czy stos nie jest pełny (np. `IsFull()`).

```
class Stack {  
    int[] items;  
    int top;  
  
    public Stack(int size) {  
        items = new int[size];  
        top = -1;  
    }  
  
    public void Push(int value) {  
        items[++top] = value;  
    }  
    public int Pop() {  
        return items[top--];  
    }  
    public int Top() {  
        return items[top];  
    }  
    public bool IsEmpty() {  
        return top == -1;  
    }  
}
```

Posługiwanie się stosem

- Mając napisaną klasę `Stack`, łatwo jest stworzyć kilka niezależnych stosów.

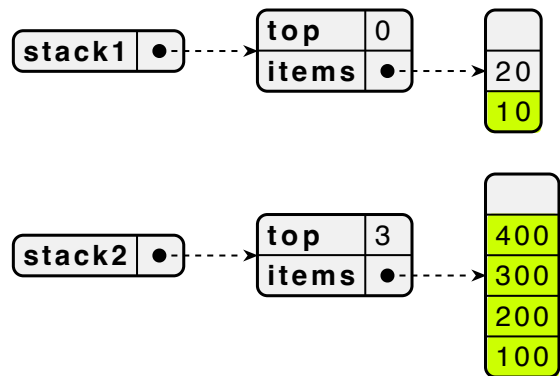
```
Stack stack1 = new Stack(3);
Stack stack2 = new Stack(5);

// Operacje na stack1
stack1.Push(10);
stack1.Push(20);

// Operacje na stack2
stack2.Push(100);
stack2.Push(200);
stack2.Push(300);
stack2.Push(400);

Console.WriteLine(stack1.Pop());
Console.WriteLine(stack2.Top());
```

Stan w pamięci po wykonaniu programu:



Stos w C++

Najważniejsze różnice

- Pole `capacity` przechowujące rozmiar tablicy (maksymalną wielkość stosu) – do ewentualnego sprawdzania przepełnienia.
- Destruktor (`~Stack()`) aby zwolnić przydzieloną dynamicznie pamięć.

```
class Stack {  
    int* items;  
    int topIndex;  
    int capacity;  
  
public:  
    Stack(int size) {  
        capacity = size;  
        items = new int[capacity];  
        topIndex = -1;  
    }  
};
```

```
~Stack() {  
    delete[] items;  
}  
  
void push(int value) {  
    items[++topIndex] = value;  
}  
  
int pop() {  
    return items[topIndex--];  
}  
  
int top() const {  
    return items[topIndex];  
}  
  
bool empty() const {  
    return topIndex == -1;  
}  
};
```

Korzystanie ze stosu w C++

- Tak zaimplementowanym stosem możemy się posługiwać jako obiektem **automatycznym**:

```
Stack stack(5);

stack.push(10);
stack.push(20);
stack.push(30);

cout << "Pop: " << stack.pop() << endl; // Usuwamy 30
cout << "Top: " << stack.top() << endl;  // 20
```

- Lub **dynamicznym**:

```
Stack *stack = new Stack(5);

stack->push(10);
stack->push(20);
stack->push(30);

cout << "Pop: " << stack->pop() << endl; // Usuwamy 30
cout << "Top: " << stack->top() << endl;  // 20

delete stack; // w tym momencie wykona się destruktor
```